Introduction to Informatics and Programming
(CIT123001)

Florian Zuleger
WT 2026/27

Chair for Fundamentals of Programming
Technical University of Munich (TUM)

From Problem to Program
A Simple Programming Language

Lecture 1

Based on Helmut Seidl's course materials, with ideas from Alexander Pretschner's slides.

Today

7. From Problem to Program

8. A Simple Programming Language

From Problem to Program

A **problem** consists in determining unknown information from given information.

A mathematical model

A problem can be represented by a **function**

$$f : I \longrightarrow O,$$

where I is the set of admissible **inputs** and O the set of possible **outputs**.

Examples and algorithms

- Addition: $f : \mathbb{Q} \times \mathbb{Q} \to \mathbb{Q}$.
- Primality testing: $f : \mathbb{N}_{>0} \to \{\text{yes}, \text{no}\}$.
- Backgammon: position and dice $\mapsto$ a move maximizing win probability.
  Win/loss scoring; optimal play by both players; fixed rule for ties.

An **algorithm** is a precise description of how to obtain the desired result from the input.

We say that the algorithm **computes** or **realizes** the function f.

Al-Khwarizmi



Muhammad ibn Musa al-Khwarizmi, 9th century

Step-by-step descriptions

An algorithm describes a sequence of individual processing steps. Familiar analogies include:

Result	Description	Steps
Sweater	Knitting pattern	Knit, purl, ...
Cake	Recipe	Take three eggs, ...
Concert	Musical score	Play the notes

Caveat: not every function is computable.

↑ **Computability Theory**

The Euclidean algorithm: the problem

Given positive integers $a, b \in \mathbb{N}_{>0}$, determine their **greatest common divisor**, $\gcd(a, b)$.

The Euclidean algorithm: the problem

Given positive integers $a, b \in \mathbb{N}_{>0}$, determine their **greatest common divisor**, $\gcd(a, b)$.

$$\begin{aligned} \gcd(5, 5) &= 5 \\ \gcd(2, 8) &= 2 \\ \gcd(30, 20) &= 10 \\ \gcd(14, 21) &= 7 \end{aligned}$$

Euclid



Euclid at work at Alexandria, around 300 BCE

The Euclidean algorithm: the steps

Input: positive integers a, b.

1. If $a = b$, stop and return a: $\gcd(a, b) = a$.
   Otherwise, continue with step 2.
2. If $a > b$, replace a by $a - b$ and return to step 1.
   Otherwise, continue with step 3.
3. Now $a < b$. Replace b by $b - a$ and return to step 1.

Why does subtraction preserve the gcd?

We show: for $a > b$, $\gcd(a, b) = \gcd(a - b, b)$.

Let $g = \gcd(a, b)$ and $g' = \gcd(a - b, b)$.

Why does subtraction preserve the gcd?

We show: for $a > b$, $\gcd(a, b) = \gcd(a - b, b)$.

Let $g = \gcd(a, b)$ and $g' = \gcd(a - b, b)$.

Factor: $a = q_a g, \quad b = q_b g$

Factor: $a - b = q'_{a-b} g', \quad b = q'_b g'$

Why does subtraction preserve the gcd?

We show: for $a > b$, $\gcd(a, b) = \gcd(a - b, b)$.

Let $g = \gcd(a, b)$ and $g' = \gcd(a - b, b)$.

Factor: $a = q_a g, \quad b = q_b g$

Subtract: $a - b = (q_a - q_b)g, \quad b = q_b g$

Factor: $a - b = q'_{a-b} g', \quad b = q'_b g'$

Add: $a = (q'_{a-b} + q'_b)g', \quad b = q'_b g'$

Why does subtraction preserve the gcd?

We show: for $a > b$, $\gcd(a, b) = \gcd(a - b, b)$.

Let $g = \gcd(a, b)$ and $g' = \gcd(a - b, b)$.

Factor: $a = q_a g, \quad b = q_b g$ **Factor:** $a - b = q'_{a-b} g', \quad b = q'_b g'$

Subtract: $a - b = (q_a - q_b) g, \quad b = q_b g$ **Add:** $a = (q'_{a-b} + q'_b) g', \quad b = q'_b g'$

This implies: g divides $a - b$ and b, so $g \leq g'$.

Similarly, g' divides both a and b, so $g' \leq g$.

Therefore $g = g'$. The case $b > a$ is symmetric.

Why does the algorithm stop?

As long as $a \neq b$, subtract the smaller positive number from the larger one.

- Both numbers remain positive.
- The positive integer $a + b$ strictly decreases.
- It cannot strictly decrease forever.

Eventually $a = b$. Since the gcd has been preserved, this common value is the gcd of the original inputs.

The assumption $a, b > 0$ matters. For example, subtracting zero would make no progress.

Properties of algorithms

Static finiteness The description has finite length. ↑ **Non-uniform Algorithms**

Dynamic finiteness At every finite point during execution, the stored intermediate information is finite.

Termination A result is obtained after finitely many steps. Reactive systems, such as operating systems, may intentionally keep running.
↑ **Operating Systems** ↑ **Reactive Systems**

Determination Equal inputs yield equal results. ↑ **Randomized Algorithms**

Determinism The next step is uniquely determined. ↑ **Nondeterministic Algorithms**

What is a program?

A **program** specifies an algorithm in a **programming language**, in a form that permits mechanical execution.

- An application often performs computations repeatedly while interacting with users or its environment.
- There are many programming languages, each suited to different tasks:
  - **Java**, Python: general-purpose programming
  - C, C++, Rust: systems programming
  - JavaScript, TypeScript: the web
  - SQL: databases
  - Haskell, Prolog: other ways of thinking about programs
  - Fortran, COBOL: decades old and still in use

What makes a language convenient?

A programming language should help

- **the programmer** express algorithmic ideas clearly and understand later what a program is supposed to do;
- **the computer** process programs and execute them efficiently.

A simple programming language

A programming language provides

- **data structures** for recording data;
- **operations** for inspecting and manipulating data;
- **control structures** for organizing these operations.

Our first example is **MiniJava**: a small teaching language using a subset of **Java** and a few input/output operations.

Variables

Variables store data so that we can retrieve it later. They must be **declared** before use.

```
int x, result;
```

This declaration introduces two variables, named `x` and `result`.

- `int` is their **type**: they store integer values.
- Names in this declaration are separated by commas.
- A semicolon ends the declaration.

Variables have values

- A variable's value can be used as an argument to an operation.
- Variables declared inside `main` get no default value: they must be assigned before their value is read. The compiler checks this.
- **Java**'s `int` values range from -2^{31} to $2^{31} - 1$. They are **not arbitrary mathematical integers**.

In the first examples, choose small inputs so that arithmetic results fit in `int`. We will return to overflow later.

Assignment

Assignment changes the value stored in a variable.

```
x = 7;
result = x;
result = x + 19;
```

- `x = 7;` stores 7 in `x`.
- `result = x;` reads `x` and stores that value in `result`.
- `result = x + 19;` computes $x + 19$ and stores the result.

In this sequence, `x` stays 7; `result` becomes 7, then 26.

Assignment reads the old value

```
result = x - 5;
x = x + 1;
```

- The first assignment subtracts `5` from the value of `x` and stores the result in `result`.
- In `x = x + 1;`, the right-hand side uses the value of `x` **before** the assignment.
- **Java** uses `=` for assignment; some languages use `:=`.
- An assignment statement ends with a semicolon.

Caveat: assignment differs from an equality test, written `==`.

Input and output

```
x = readInt();
write(42);
write(result);
write(x - 14);
```

- `readInt()` reads text from the terminal and interprets it as an integer. The assignment stores that integer in `x`.
- `write(...)` evaluates its argument and prints its value.
- `write(x - 14)` does not change `x`.

These operations are supplied by our `MiniJava` helper class.

Printing numbers and text

To print an integer, its value is converted to a sequence of characters. Our helper performs that conversion for us.

We also allow strings as arguments:

```
write("Hello World!");
```

This prints `Hello World!` followed by a line break.

A **string** is text enclosed in double quotation marks.

Sequence

```
int x, y, result;
x = readInt();
y = readInt();
result = x + y;
write(result);
```

Read two numbers, add them, and print the result.

Executing a sequence

For this straight-line program, assuming valid input and normal execution:

- One operation executes at a time.
- Each operation executes exactly once.
- Operations execute in the order in which they occur in the program.
- Execution ends after the final operation.

$\Rightarrow$ **Sequence alone gives us only a fixed list of actions.**

Selection: conditional choice

```
int x, y, result;
x = readInt();
y = readInt();
if (x > y)
    result = x - y;
else
    result = y - x;
write(result);
```

- Evaluate the condition.
- If true, execute the statement after `if`.
- Otherwise, execute the statement after `else`.
- Continue with `write(result)`.

Conditions compare values: == (equal), != (not equal), <, <=, >, >=.

Alternatives can contain statements

```
int x;
x = readInt();
if (x == 0)
    write(0);
else if (x < 0)
    write(-1);
else
    write(+1);
```

The second alternative contains another `if` statement.

The program prints the sign of the input: -1, 0, or $+1$.

The else part may be absent

```
int x;
x = readInt();
if (x < 0)
    x = -x;
write(x);
```

If the condition is false, nothing happens: continue with `write(x)`.

The program prints the absolute value of the input.

Blocks group statements

Curly braces group statements into a **block**. Both programs have just read `x`. What if we leave out the braces?

```
if (x != 0) {
    y = readInt();
    if (x > y)
        write(x);
    else
        write(y);
} else
    write(0);
```

Without braces: does not compile: the last `else` has no `if`.

```
if (x >= 0) {
    if (x > 100)
        write("large");
} else
    write("negative");
```

Without braces: compiles, but the `else` belongs to the **nearest** `if`: 0 to 100 print `negative`.

$\Rightarrow$ **Java ignores indentation; braces decide the structure.**

Iteration: repeated execution

With sequence and selection, each statement runs at most once. A `while` loop repeats statements.

```
int x, y;
x = readInt();
y = readInt();
while (x != y)
    if (x < y)
        y = y - x;
    else
        x = x - y;
write(x);
```

This is the subtraction-based Euclidean algorithm.

Precondition: both input values are positive integers.

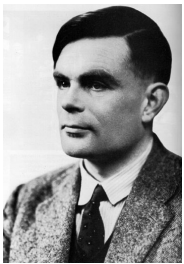
When the loop stops, `x == y`; print this common value.

How a while statement executes

1. Evaluate the condition.
2. If true, execute the **body**, then return to step 1.
3. If false, continue after the `while` statement.

The body can execute zero times: the condition is checked before the first iteration.

How powerful is MiniJava?



Alan Turing, 1912–1954

1936: Turing introduces a simple model of computation: the **Turing machine**. A function is **computable** if a Turing machine can compute it.
↑ **Computability Theory**

A surprising result: MiniJava can simulate a Turing machine using sequence, selection, and iteration, if integers are **unbounded** (only two are needed).

⇒ **Three control structures suffice for every computation.**

Enough in principle, with limits

Idealized Real **Java** integers are finite; the result assumes integers of any size.

Partial **Java** programs may run forever without output, so they compute **partial** functions. Our gcd example loops for inputs 0 and 5 at $y = y - 0$.

Limits Some problems no program can solve, such as deciding whether an arbitrary program stops: the **halting problem**.

$\Rightarrow$ **MiniJava matches Turing-machine power, but some problems are unsolvable.**

From MiniJava to executable Java

Our MiniJava examples use an explicit **Java** class and a `main` method.

Our running example is the gcd program:

```
int x, y;
x = readInt();
y = readInt();
while (x != y)
    if (x < y)
        y = y - x;
    else
        x = x - y;
write(x);
```

GCD.java: the added Java wrapper

```
public class GCD extends MiniJava {
    public static void main(String[] args) {
        int x, y;
        x = readInt();
        y = readInt();
        while (x != y)
            if (x < y)
                y = y - x;
            else
                x = x - y;
        write(x);
    }
}
```

What the surrounding code means

- The public class is named `GCD`; its file is `GCD.java`.
- Our MiniJava instructions form the body of the `main` method.
- For this program, the **Java** launcher calls `public static void main(String[] args)`.
- The helper class `MiniJava` supplies `readInt()` and `write()`.
- `extends MiniJava` makes these operations available in `GCD`.

We will study classes, inheritance, and the keywords `public` and `static` later.

MiniJava.java: displaying results (1/2)

```
public class MiniJava {
    public static void write(String x) { IO.println(x); }
    public static void write(int x) { IO.println(x); }
```

MiniJava programs use `write(...)` to display their results.

`IO.println(...)` prints a value and starts a new line.

MiniJava.java: reading input (2/2)

```
    public static String readString() { return IO.readln(); }

    public static int readInt() {
        return Integer.parseInt(readString().trim());
    }
}
```

`IO.readln()` reads text; `parseInt` converts it to an integer.

`readInt()` reads one integer per line for our examples.

`main` without an explicit class

Java 25 also allows a program without `class GCD extends MiniJava`:

```
void main() {
    // Precondition: positive inputs.
    int x, y;
    x = readInt();
    y = readInt();
    while (x != y)
        if (x < y)
            y = y - x;
        else
            x = x - y;
    write(x);
}
```

In the same file:

```
int readInt() {
    return Integer.parseInt(
    IO.readln().trim());
}

void write(String x) {
    IO.println(x);
}

void write(int x) {
    IO.println(x);
}
```

Why we keep MiniJava and an explicit class

Behind the scenes: the compact source file implicitly declares a class containing `main()`, `readInt()`, and both `write` methods.

To run this program, the launcher creates an object of that class and calls its `main()` method.

Our course: we keep MiniJava and the explicit `GCD` class, so the program's structure and its helper operations remain visible.

$\Rightarrow$ **We will explain classes, objects, and what the launcher does later.**

Comments explain a program

Comments help a reader understand the code.

```
// This comment extends to the end of the line.

/* This comment extends
   across multiple lines. */
```

Use comments to explain intentions, assumptions, and non-obvious choices.

Compile and run

In a terminal, in the directory containing the **Java** source files:

```
javac GCD.java MiniJava.java
java GCD
```

- `javac` compiles the sources to JVM bytecode in `GCD.class` and `MiniJava.class`.
- `java` calls the `main` method of class `GCD`.
- Enter `30`, press Enter; enter `20`, press Enter. The program prints `10`.

Let the compiler find dependencies

In the directory containing both source files, you can also run:

```
javac GCD.java
java GCD
```

- With default search settings, `javac` finds `MiniJava.java` automatically in the same directory.
- `GCD.java` is compiled. `MiniJava.java` is compiled if `MiniJava.class` is missing or older; otherwise the class file is reused.
- Explicitly listing both source files recompiles both, regardless of existing `.class` timestamps.

An important extension: arrays

Arrays store sequences of values, accessed by their **indices**.

```
int[] a = new int[100];
```

This creates 100 integer elements, from `a[0]` to `a[99]`.

An important extension: arrays

Arrays store sequences of values, accessed by their **indices**.

```
int[] a = new int[100];
```

This creates 100 integer elements, from `a[0]` to `a[99]`.

```
int i;
a[5] = 1;          // a[5] now contains 1
i = 5;
a[i] = 7;          // a[5] now contains 7
i = 10;
a[i] = 42;         // a[10] now contains 42
a[i+1] = a[i] + 2; // a[11] now contains 44
```

Array indices start at zero

- `a.length` is the number of elements: here, 100.
- Valid indices satisfy $0 \leq i < \texttt{a.length}$.
- Thus `a[99]` is the final element; **`a[100]` is out of bounds**.
- **Java** checks indices at run time. An invalid index causes an `ArrayIndexOutOfBoundsException`.
- `new int[100]` sets every element to 0. Unlike local variables, array elements can be read before any assignment.

We will return to arrays and exceptions in more detail later.

MiniJava is only the beginning

MiniJava, even with arrays, is a small language.

Java offers many additional features to help us write programs:

- more types and data structures than `int` and `int[]`;
- more control structures;
- mechanisms for organizing larger programs.

These will be introduced later in the course.